

Final Project

Churn Prevention System

Table of Contents

1. Project Objective	2
2. Dataset	2
3. Problem Formulation	3
4. Common Pipeline	3
5. System Diagram Requirement	6
6. Engineering Requirements	7
7. Expected Outcome	7

1. Project Objective

The goal of this project is to build a complete **end-to-end Churn Prevention System** that combines:

- **Churn Prediction:** Identify customers who are likely to stop purchasing
- **Recommendation System:** Suggest personalized products to retain those customers

You will implement a full pipeline from **raw transactional data** → **modeling** → **serving** → **evaluation**, simulating a real-world production ML system.

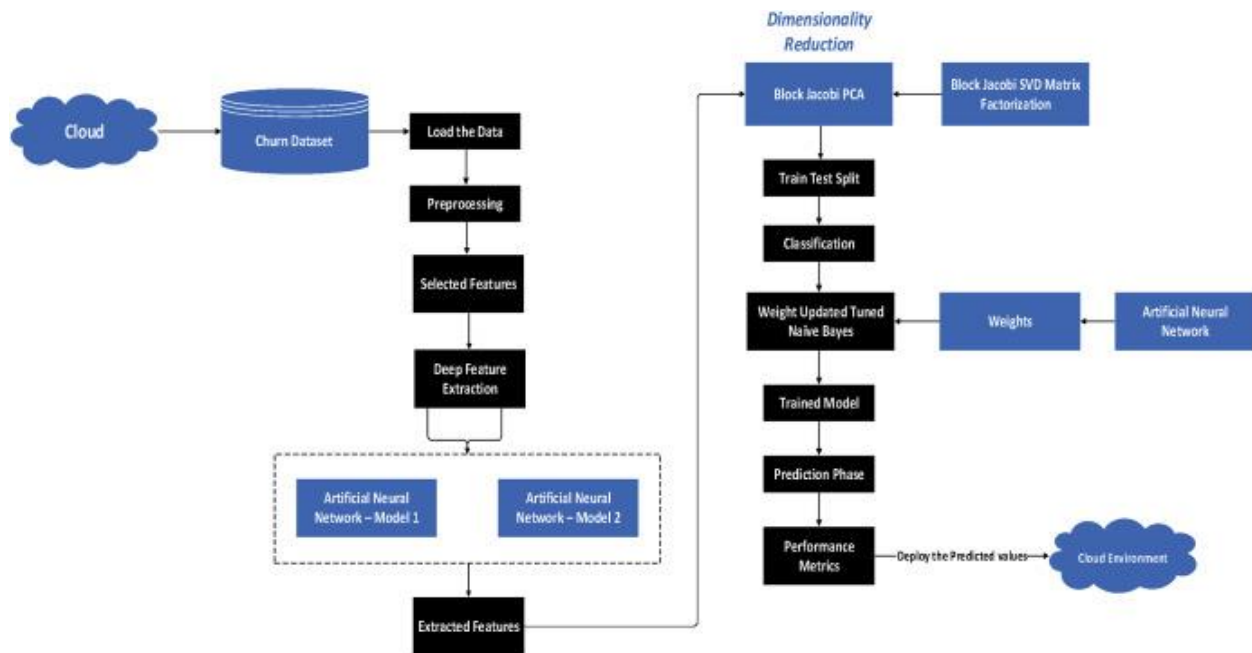
The system should include:

- **Data preprocessing & feature engineering**
- **Churn prediction model**
- **User-item representation learning**
- **Retrieval + ranking recommendation system**
- (Optional) LLM-based reranker
- **REST API serving**
- **Offline evaluation & business analysis**

The final system must simulate a **production-ready ML system**, including:

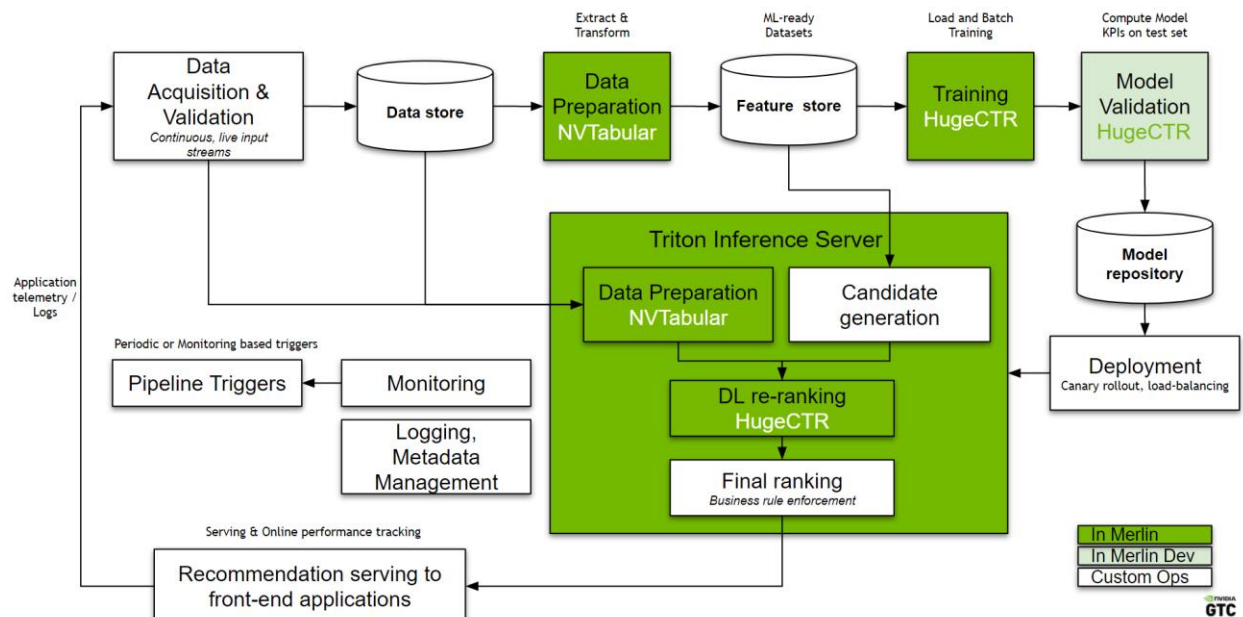
- CI/CD pipeline (optional)
- **A complete system architecture diagram (Draw.io / Excalidraw)**
- **Clear separation between offline training and online inference**

Example of required Churn Diagram:



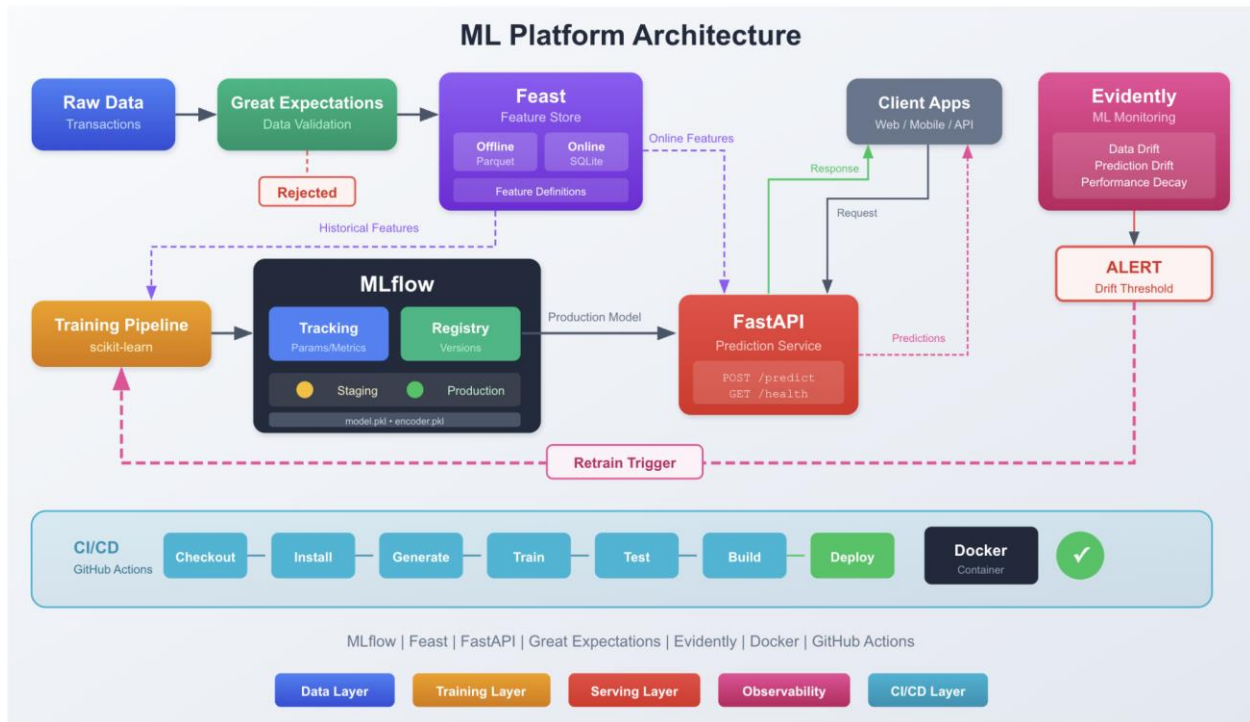
Source: <https://www.sciencedirect.com/science/article/abs/pii/S0952197623011995>

Example of required Recommendation Diagram:



Source: <https://developer.nvidia.com/blog/continuously-improving-recommender-systems-for-competitive-advantage-with-merlin-and-mlops/>

Example of required CI/CD deployment:



Source: <https://www.freecodecamp.org/news/build-end-to-end-ml-platform-locally-from-experiment-tracking-to-cicd/>

Note: The expected diagram should be a combination of the 3 diagrams above.

2. Dataset

Dataset Used: Online Retail II Dataset

Download Link: <https://www.kaggle.com/datasets/mashlyn/online-retail-ii-uci>

Dataset Description

This dataset contains real-world **e-commerce transaction data** from a UK-based online retailer.

Main fields:

- CustomerID: unique customer identifier
- InvoiceDate: timestamp of transaction
- StockCode: product ID
- Quantity: number of items purchased

- Price: unit price

3. Problem Formulation

3.1 Churn Definition

Since the dataset does not include churn labels, you must **define churn based on behavior**.

- A customer is considered churned if they have no purchase within the next 90 days after their last transaction.

3.2 Label Construction

For each customer:

1. Sort transactions by time
2. Identify last purchase date in training window
3. Check if the customer returns within next **N days (e.g., 90 days)**
 - If NO → churn = 1
 - If YES → churn = 0

3.3 Feature Engineering (for Churn Model)

- **Recency**: Days since last purchase
- **Frequency**: Number of transactions
- **Monetary**: Total spending
- Behavioral features:
 - Avg basket size
 - Purchase interval
 - Product diversity

4. Common Pipeline

4.1 Data Pipeline

- Clean transactional data
- Remove invalid records (returns, null CustomerID, etc.)
- Generate:
 - Churn dataset (user-level)
 - Interaction dataset (user–item level)
- Perform **time-based train/validation/test split**

4.2 Churn Prediction Model

Train a model to predict churn probability.

Supported models:

- Churn Classification
- BG-NBD
- Survival Analysis

Output:

- Churn probability score per user
- Or p-alive, associated with time to churn

4.3 Retrieval Model (Recommendation Candidate Generation)

Build a model to learn **user and item embeddings**.

Supported approaches:

- ALS
- Item2Vec
- Two-Tower model
- Graph-based recommendation

Output:

- User embedding
- Item embedding
- Top-N candidate items

4.4 Approximate Nearest Neighbor Search (FAISS)

- Build FAISS index from item embeddings
- Retrieve Top-N items for each user

4.5 Ranking Model (Churn-Aware Recommendation)

Train a ranking model to prioritize the best products for retention.

Input features:

- User embedding
- Item embedding
- Churn probability

- Interaction features

Supported models (one of these models):

- BPR
- NeuMF
- Wide & Deep / DeepFM
- Sequential models

Output:

- Ranked Top-K recommendations

4.6 Optional: LLM-based Reranker

Use an LLM to refine recommendations.

Input:

- User behavior summary
- Product metadata

Output:

- Semantically improved ranking

4.7 Decision Layer (Retention Strategy)

Combine:

- Churn probability
- Recommended products

To produce:

- Final retention action (e.g., product suggestion, discount targeting)

4.8 API Serving

Build a REST API:

- input user → churn prediction → retrieval → ranking → output Top-K products

Requirements:

- At least one inference endpoint

- Use FastAPI
- Dockerize the system

4.9 Evaluation Report

Churn Model Metrics

- AUC-ROC
- Precision / Recall / F1

Recommendation Metrics

- Recall@K
- NDCG@K

Business Evaluation

- Performance on high-churn segment
- Recommendation impact on retention

Ablation Study

Compare:

- Churn only
- Churn + Retrieval
- Churn + Retrieval + Ranking
 - LLM (optional)

5. System Diagram Requirement

You must provide a system architecture diagram including:

Offline Pipeline

- Data preprocessing
- Feature engineering
- Churn model training
- Embedding training
- FAISS indexing

Online Pipeline

- API request
- Churn scoring

- Candidate retrieval
- Ranking
- Response generation

Clearly separate:

- Training phase
- Inference phase

6. Engineering Requirements

Suggested Project Structure

- data/
- features/
- models/
 - churn/
 - retrieval/
 - ranking/
- faiss/
- api/
- evaluation/

Technical Requirements

- PyTorch for modeling
- FAISS for ANN search
- FastAPI for serving
- Docker for deployment

Code Quality

- Reproducible pipeline
- Well-documented code
- Publish on GitHub

7. Expected Outcome

By completing this project, you will:

- Understand how to combine **churn prediction + recommendation systems**
- Learn how to define **labels from raw behavioral data**
- Build a **production-style ML system**

- Gain hands-on experience with **end-to-end system design**